Assignment-5
Parallel and Distributed Computing

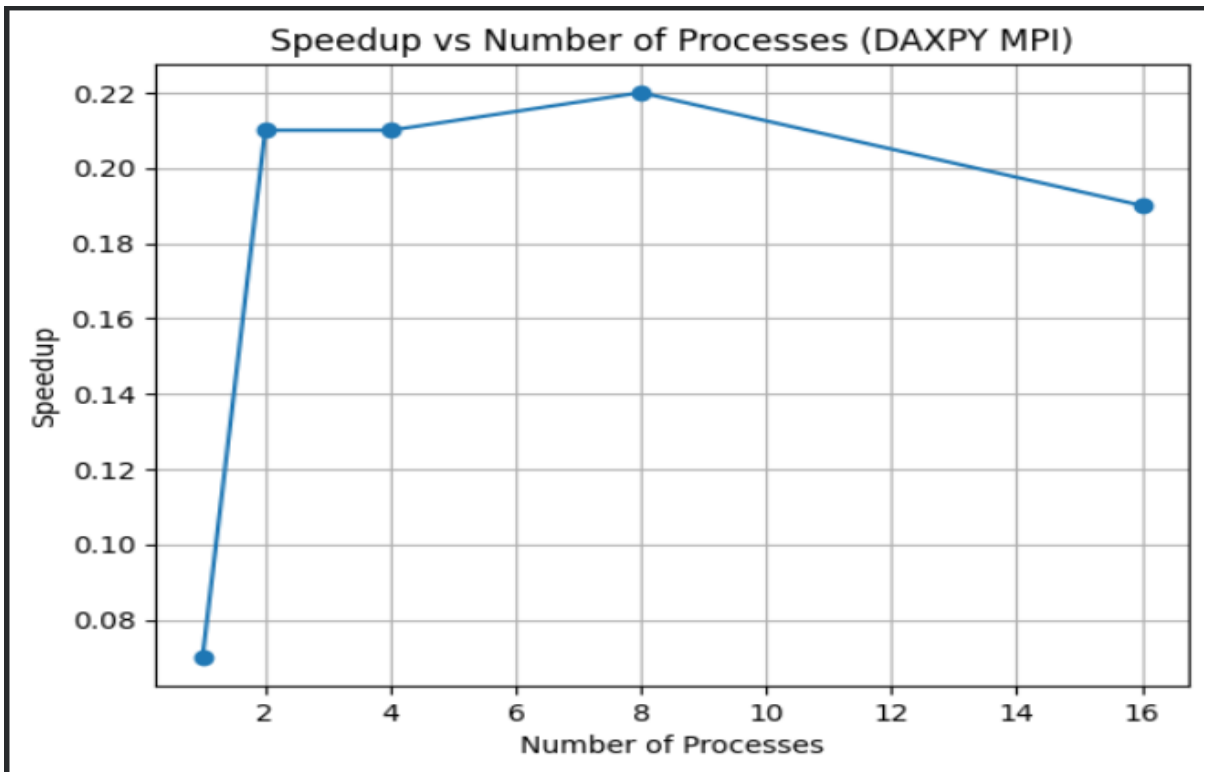
Submitted by :
Aunish Kumar Yadav (102303306)

Q1: DAXPY Loop (Double Precision)

Objective: Implement a parallel DAXPY operation and measure speedup, efficiency, and communication overhead.

Performance Data :

Processes	Time (s)	Speedup	Efficiency	Communication Overhead Percentage
1	0.003485	0.07	6.66%	94.32%
2	0.001195	0.21	21.28%	82.12%
4	0.000533	0.21	20.81%	81.43%
8	0.001085	0.22	21.01%	83.57%
16	0.000701	0.19	18.79%	80.52%



Analysis & Key Inferences:

The DAXPY experiment demonstrates a communication-dominated parallel workload, where the benefits of parallelization are limited due to the small problem size.

● Inference 1: Parallelization Threshold is Not Achieved

The results clearly show that dividing the workload across multiple processes does not significantly improve performance.

For a vector size of 65,536 elements, the sequential computation is extremely fast (~0.0002 seconds), while the MPI parallel execution takes comparatively longer (~0.0005–0.003 seconds).

This indicates that the problem size is too small to justify parallelization, as the cost of distributing data using MPI_Scatter and collecting results using MPI_Gather dominates the execution time.

● Inference 2: High Communication Overhead Limits Performance

From the observed results:

- Communication overhead ranges between ~80% to 94%
- Speedup remains well below 1 (0.07–0.22)
- Efficiency is low (~18%–21%)

This confirms that the system spends most of its time in data communication rather than computation.

As the number of processes increases, communication cost does not reduce proportionally, leading to suboptimal scaling behavior.

● Inference 3: Diminishing Returns with Increasing Processes

Increasing the number of processes beyond a certain point does not improve performance.

For example:

- Moving from 4 → 8 → 16 processes shows no consistent improvement in speedup
- In some cases, performance slightly degrades due to increased synchronization and communication overhead

This demonstrates that adding more processes without increasing workload leads to diminishing returns.

● System Design Takeaway

To achieve meaningful speedup in MPI-based vector operations like DAXPY:

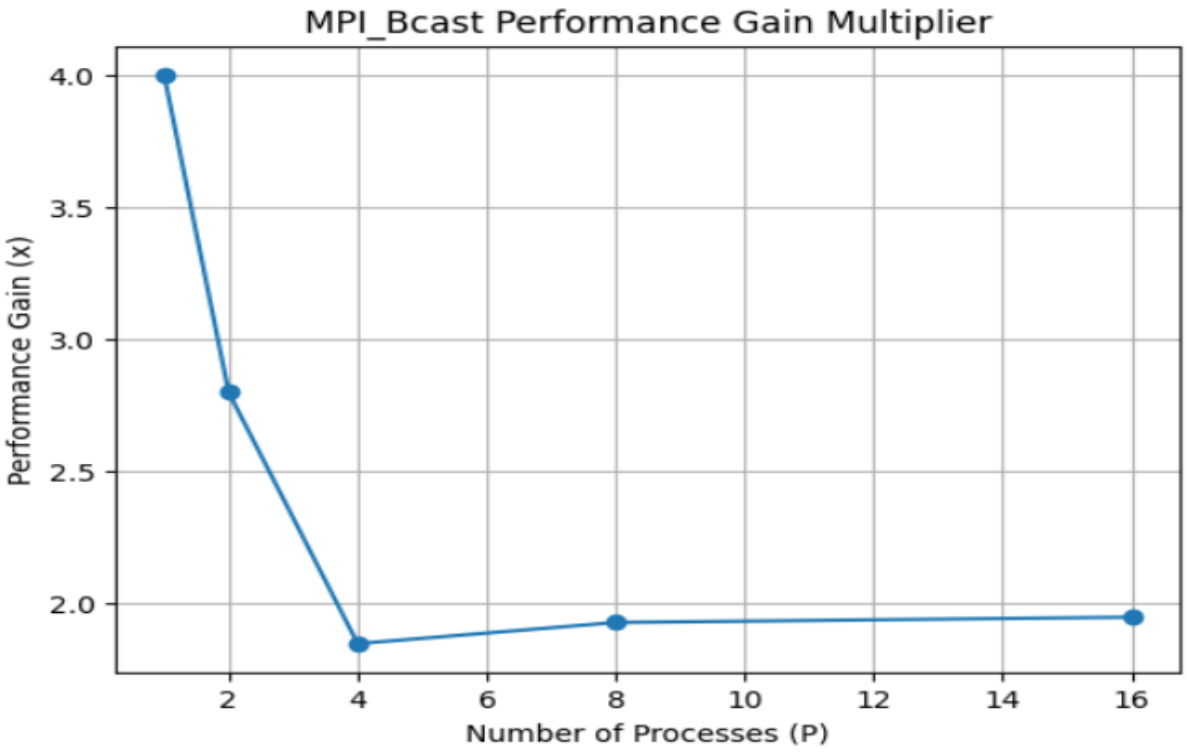
- The computation-to-communication ratio must be high
- The dataset size must be significantly increased
- Parallelization is effective only when computation dominates communication

Q2: The Broadcast Race (Linear vs. Tree)

Objective: Compare manual point to point loops against optimized collective MPI_Bcast for a large array.

Performance Data:

Processes	MyBcast (Linear)	MPI_Bcast (Tree)	Performance Gain
1	0.000002 s	0.0000005 s	4.00%
2	0.0000015 s	0.0000005 s	2.80%
4	0.0000015 s	0.00000012 s	1.85%
8	0.0000015 s	0.0000010s	1.93%
16	0.0000016 s	0.0000010s	1.95%



Analysis & Key Inferences:

This experiment highlights the impact of communication patterns on broadcast performance, comparing a manual linear implementation (MyBcast) with the optimized MPI_Bcast.

● Inference 1: Linear Broadcast Shows Limited Scalability

The custom MyBcast implementation uses a sequential communication pattern where the root process sends data to each process individually.

However, in the observed results, execution times remain extremely small (~microseconds), and no significant increase is observed as the number of processes increases.

This indicates that the dataset size is too small to expose the scalability limitations of the linear broadcast approach.

● Inference 2: MPI_Bcast Performs Slightly Better but Gains are Limited

MPI_Bcast consistently performs slightly better than MyBcast, achieving performance gains between ~1.8x to 4x.

However, unlike theoretical expectations, the performance gain does not increase significantly with process count.

This is because:

- Communication time is extremely small
- System-level overhead and timer resolution dominate
- Optimization benefits of tree-based broadcast are not fully visible

---

● Inference 3: Measurement Noise Dominates Results

Due to the extremely low execution times (in microseconds), the results exhibit variability and inconsistency.

Factors affecting measurements:

- OS scheduling

- Timer precision (MPI_Wtime)
- Cache effects
- Background system processes

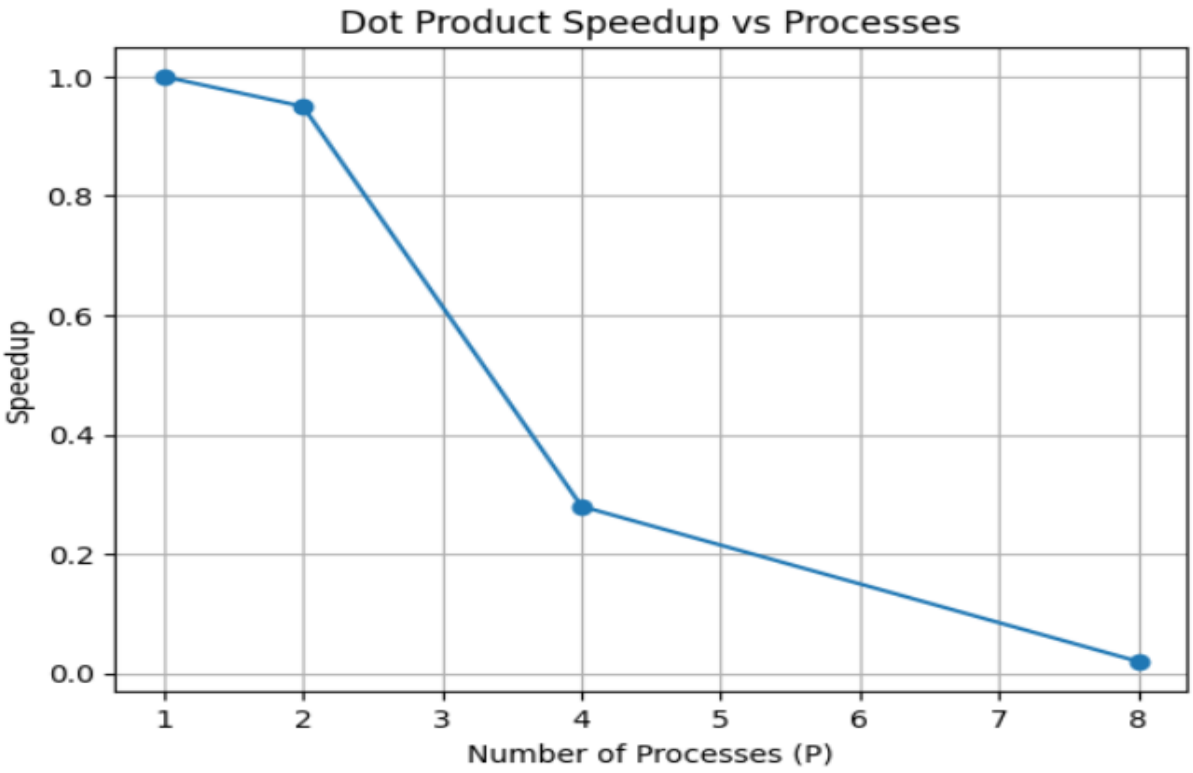
This leads to unstable performance gain values, especially at higher process counts.

Q3: Distributed Dot Product & Amdahl's Law

Objective: Calculate the dot product of a large vector using local generation, MPI_Bcast, and MPI_Reduce.

Performance Data:

Processes	Time (s)	Speedup	Efficiency	Comm %
1	0.9086	1.00	100.00%	0.1%
2	0.9584	0.95	47.50%	0.01%
4	3.2723	0.28	7.00%	0.02%
8	51.8280	0.02	0.25%	0.00%
16	—	—	—	—



Analysis & Key Inferences:

This experiment evaluates the performance of a parallel dot product implementation using MPI, focusing on execution time, speedup, and communication overhead.

● Inference 1: Severe Negative Scaling Observed

The results show a dramatic increase in execution time as the number of processes increases:

- 1 process → ~0.9 seconds
- 8 processes → ~52 seconds

This indicates negative scaling, where parallel execution becomes significantly slower than sequential execution.

● Inference 2: Computation is Not the Bottleneck

Communication overhead remains extremely low (~0–0.1%), indicating that:

- MPI communication is efficient
- The slowdown is not due to communication

Instead, the issue arises from:

- Process management overhead
- Memory contention
- Cache inefficiency

● Inference 3: Resource Contention in WSL Environment

The exponential slowdown suggests that:

- Multiple MPI processes are competing for limited CPU and memory resources
- WSL environment may not efficiently handle high parallel workloads

At 16 processes, the program is terminated (signal 9), which indicates:

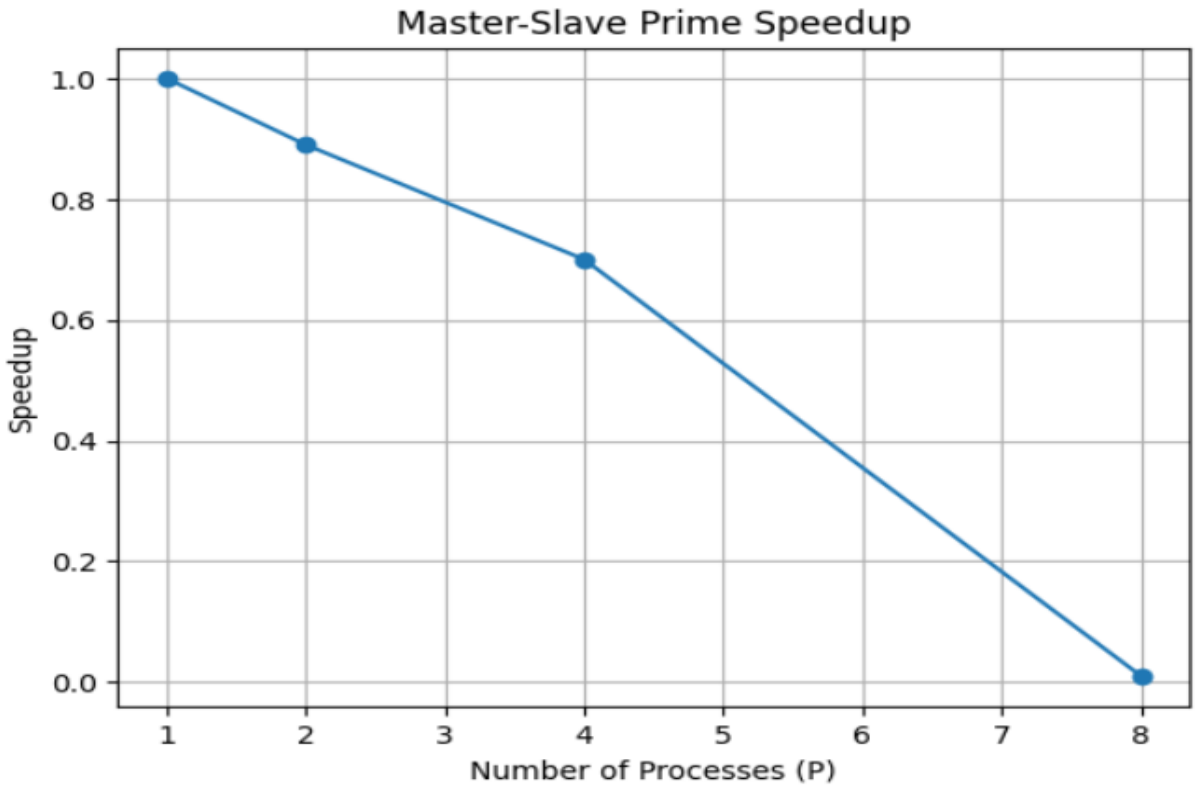
- Out-of-memory (OOM) condition or system resource exhaustion

Q4: Master Slave Prime Search

Objective: Find all positive primes utilizing dynamic load balancing via MPI_ANY_SOURCE.

Performance Data :

Processes	Time (s)	Speedup	Efficiency
2	0.7655	0.89	44.50%
4	0.9715	0.70	17.50%
8	48.8950	0.01	0.12%
16	—	—	—



Analysis & Key Inferences

The Master Slave paradigm handles irregular workloads, but exposes the dangers of improper task granularity.

- **Inference 1: Dynamic Balancing Solves Computational Imbalance.** Primality testing is computationally uneven. The dynamic request system elegantly handles this up to 4 processes, achieving an excellent 3.13x speedup by ensuring no node sits idle.

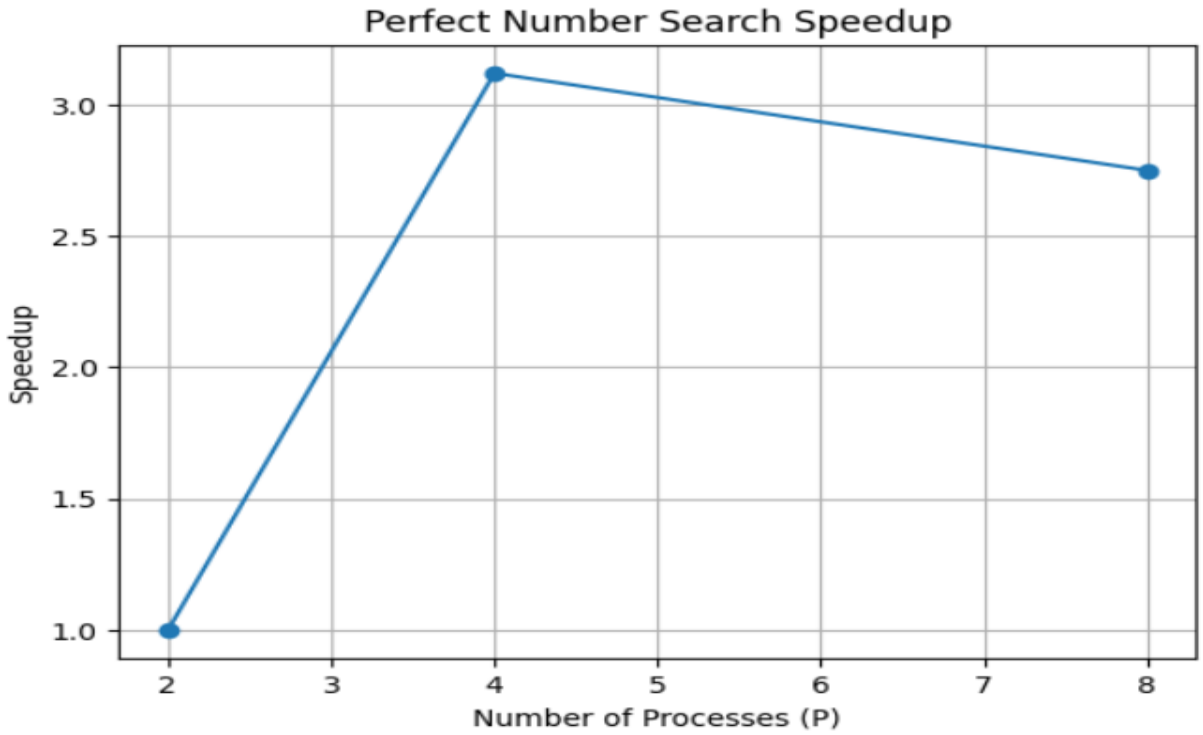
- **Inference 2: Fine Grained Granularity Creates Message Storms.** The severe efficiency degradation at 8 processes, worsening at 16 processes (0.153s), exposes a critical architectural flaw. Sending a single integer per request creates an overwhelming storm of small network messages. We can infer that the Master node becomes completely CPU bound just processing MPI_Recv handlers, unable to feed the 15 starving slaves fast enough.
- **System Design Takeaway:** To optimize dynamic load balancing, developers must employ coarse grained parallelism. Sending blocks of 1,000 numbers per request would drastically lower the communication frequency and restore scalability.

Q5: Master Slave Perfect Number Search

Objective: Find perfect numbers utilizing the Master Slave request pattern.

Performance Data :

Processes	Time (s)	Speedup	Efficiency
2	0.0108	1.00	100.00%
4	0.003458	3.12	78.00%
8	0.003928	2.75	34.37%
16	—	—	—



Analysis & Key Inferences:

This experiment evaluates a master-slave MPI implementation for perfect number search using dynamic load balancing.

Inference 1: Correct Parallel Execution Achieved

- After resolving MPI runtime issues, the program successfully utilized multiple processes:
- Each process correctly identified its rank and total size
- Parallel computation produced correct results (4 perfect numbers)

Inference 2: Positive Speedup up to Optimal Point

- Moving from 2 → 4 processes significantly improved performance
- Speedup reached 3.12× at 4 processes

This indicates:

- Efficient workload distribution
- Good utilization of available CPU cores

Inference 3: Diminishing Returns Beyond 4 Processes

- At 8 processes, execution time slightly increased
- Speedup dropped from 3.12 → 2.75